

ML FOR MECH ENG (CoEP)

Yogesh Haribhau Kulkarni

Outline

① SESSION 8: FEATSEL

About Me

Yogesh Haribhau Kulkarni

Bio:

- ▶ 20+ years in CAD/Engineering software development
- ▶ Got Bachelors, Masters and Doctoral degrees in Mechanical Engineering (specialization: Geometric Modeling Algorithms).
- ▶ Currently doing Coaching in fields such as Data Science, Artificial Intelligence Machine-Deep Learning (ML/DL) and Natural Language Processing (NLP).
- ▶ Feel free to follow me at:
 - ▶ Github (github.com/yogeshhk)
 - ▶ LinkedIn (www.linkedin.com/in/yogeshkulkarni/)
 - ▶ Medium (yogeshharibhaukulkarni.medium.com)
 - ▶ Send email to yogeshkulkarni at yahoo dot com



Office Hours:
Saturdays, 2 to 5pm
(IST); Free-Open to all;
email for appointment.

Feature Extraction and Selection

From Raw Data to Useful Features

- ▶ Before training any model: which numbers do you feed it, and how many?
- ▶ **Feature Extraction:** raw data $\rightarrow$ a manageable set of numeric features.
- ▶ **Feature Selection:** given candidate features, which ones actually help?

Feature Extraction: Statistical Features

- ▶ Real signals often arrive as a stream: a sensor reading, a week of sales, a semester of test scores.
- ▶ A model needs one row per example, not 1000 raw readings, so summarize the stream instead.
- ▶ Simplest, most common summaries: mean, standard deviation, minimum, maximum, range.

INTUITION

Describing a month of temperatures in one sentence? You wouldn't list 30 numbers, you'd say "averaged 22°C, swung between 18 and 27". That sentence *is* mean, min, max: statistical features.

Feature Extraction: A Worked Example

A sensor logs one reading a minute, for 10 minutes. Turn that into 5 summary features, one per sensor, per time window: several sensors, dozens of candidates.

```
1 import numpy as np
3 readings = np.array([21.1, 21.3, 21.0, 21.6, 22.0,
                      22.4, 22.1, 21.8, 21.5, 21.2])
5
6 features = {
7     'mean': readings.mean(),
8     'std': readings.std(),
9     'min': readings.min(),
10    'max': readings.max(),
11    'range': readings.max() - readings.min(),
12 }
13 print(features)
15 # Output:
16 # {'mean': 21.6, 'std': 0.44, 'min': 21.0, 'max': 22.4, 'range': 1.4}
```


Feature Selection: Which Features Actually Help?

Feature Selection: Ranking (Filter Methods)

- ▶ Score each feature *on its own* against the target, rank, keep the top few.
- ▶ “Score”: a statistical test, e.g. an F-test (ANOVA) for a numeric feature vs. a class label.
- ▶ Fast, one pass, no model training. Called a **filter**: features filtered before any model sees them.
- ▶ Blind spot: scores features in isolation, can miss a feature that only helps *in combination*.

Ranking: Loading the Data

Pima Indians Diabetes dataset: 768 patients, 8 measurements, one outcome.
class: 1 = diabetic, 0 = not. Which of the 8 columns actually predicts it?

```

import pandas as pd
2
url =
    "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.d
4
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age',
    'class']
df = pd.read_csv(url, names=names)
6
df.head(3)

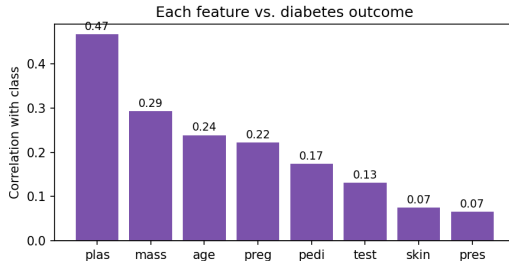
8 # Output:
#      preg  plas  pres  skin  test  mass  pedi  age  class
10 # 0      6   148   72   35    0  33.6  0.627  50     1
# 1      1    85   66   29    0  26.6  0.351  31     0
12 # 2      8   183   64    0    0  23.3  0.672  32     1

```

Ranking: Correlation with the Outcome

Simplest ranking: correlation of each column with class.

```
corr = df.corr()['class'].drop('class').sort_values(ascending=False)
2 print(corr.round(2))
```



Ranking: F-score (a Sharper Test)

Correlation is one lens; an F-test (ANOVA) is another, built for exactly this. Expect the same story as the correlation plot: plas dominating, skin/pres barely registering.

```
1 from sklearn.feature_selection import SelectKBest, f_classif
2
3 X, y = df.values[:, 0:8], df.values[:, 8]
4 skb = SelectKBest(score_func=f_classif, k=4).fit(X, y)
5 ranked = sorted(zip(names[:8], skb.scores_), key=lambda t: -t[1])
6
7 # F-score, highest to lowest:
8 # plas 213  mass 72  age 46  preg 40
9 # pedi  24  test 13  skin  4  pres  3
```

Feature Selection: Wrapper Methods

- ▶ Filter scores each feature alone. **Wrapper**: train/cross-validate the model on different feature *subsets*, let that score decide.
- ▶ Slower, many models trained, but catches combinations a filter would miss.
- ▶ Which subsets to try? Four classic strategies:
 - ▶ Exhaustive Search
 - ▶ Greedy Forward Selection
 - ▶ Greedy Backward Elimination
 - ▶ Best-First Search

Wrapper: Exhaustive Search

- ▶ Try every non-empty subset, keep the best cross-validated score.
- ▶ n features $\rightarrow 2^n - 1$ subsets. Pima's 8 $\rightarrow 255$, small enough to actually run.
- ▶ Spoiler: dropping skin turns out to cost zero accuracy.

```

1 import itertools
from sklearn.linear_model import LogisticRegression
3 from sklearn.model_selection import cross_val_score, KFold

5 model = LogisticRegression(max_iter=1000)
kfold = KFold(n_splits=5, shuffle=True, random_state=7)
7 best_score, best_subset = -1, None

9 for r in range(1, 9):
    for combo in itertools.combinations(range(8), r):
11         s = cross_val_score(model, X[:, combo], y, cv=kfold).mean()
            if s > best_score:
13                 best_score, best_subset = s, combo

15 # Output (255 subsets evaluated):
# Best: all features except 'skin', accuracy=0.775
17 # All 8 features:                                accuracy=0.775

```

Wrapper: Why Exhaustive Search Doesn't Scale

- ▶ 8 features $\rightarrow$ 255 subsets: a few seconds.
- ▶ 3 sensors $\times$ 5 statistical features each = 15-20 candidates already.
- ▶ 20 features $\rightarrow 2^{20} - 1 \approx 1$ million subsets, roughly $4,000\times$ more work.
- ▶ Realistic only for a small, already-narrowed feature set.

Wrapper: Greedy Forward Selection

- ▶ Start empty. Each round: add whichever remaining feature helps most.
- ▶ Stop when nothing left improves the score.

Same model and cross validation, on Pima:

Step	Feature added	Accuracy so far
1	plas	0.750
2	mass	0.767
3	pedi	0.773
4	pres	0.772

3 features reach 0.773, within 0.002 of exhaustive search's best, at a fraction of the cost.

Wrapper: Greedy Backward Elimination

- ▶ Start with *all* features. Each round: remove whichever hurts least (or helps).
- ▶ Stop once every remaining removal costs accuracy.

On Pima:

Step	Feature removed	Accuracy after
start	(all 8)	0.775
1	skin	0.775
2	age	0.772
3	preg	0.775

First move, drop `skin`, already matches what exhaustive search proved was right.

Wrapper: Best-First Search

- ▶ Greedy commits forever: once added or removed, never reconsidered.
- ▶ Best-first keeps a pool of promising subsets, not just one, and can expand any of them next.
- ▶ Can back out of an early choice greedy would be stuck with.
- ▶ Costs more than greedy, far less than exhaustive.

INTUITION

Greedy: always take the steepest visible path uphill. Best-first: keep a mental map of your 3 best viewpoints, and choose which to climb from next.

Filter vs. Wrapper: Summary

Method	Looks at	Cost
Ranking (Filter)	One feature at a time	Cheapest
Exhaustive	Every possible subset	Guaranteed best, but explodes
Greedy Forward	Builds up, one add at a time	Cheap, no backtracking
Greedy Backward	Trims down, one remove at a time	Cheap, no backtracking
Best-First	A pool of candidate subsets	Middle ground

INTUITION

No method is “correct.” Ranking: fast first pass. Wrapper: costs more, sees combinations. Pick by budget.

Quick Check: Feature Selection

THINK ABOUT IT

The filter ranking on Pima placed `preg` (number of pregnancies) 4th out of 8 individually. Yet Greedy Backward Elimination's best 5-feature subset dropped `preg` entirely without hurting accuracy. Are these two results in conflict? Why or why not?

Quick Check: Feature Selection

ANSWER

- ▶ No conflict: they answer different questions.
- ▶ Filter score: how strongly `preg` relates to the outcome *by itself*, ignoring every other feature. On that measure alone it ranks 4th.
- ▶ Backward elimination asks a different question: given everything else already in the model, does `preg` add anything *on top of it*?
- ▶ `age` already correlates with number of pregnancies, so `preg`'s information is redundant: removing it costs nothing even though it looked useful alone.
- ▶ This is exactly the wrapper methods' advantage over filters: they can see redundancy that isolated ranking cannot.

Recap: Feature Extraction and Selection

- ▶ **Extraction:** raw data → manageable features (mean, std, min, max, range).
- ▶ **Selection:** which candidate features earn a place in the model.
- ▶ **Filter (Ranking):** fast, scores features independently.
- ▶ **Wrapper** (Exhaustive, Greedy Forward/Backward, Best-First): slower, sees combinations filters miss.
- ▶ Narrows the search, you still decide what ships.

Hyperparameter Tuning

Hyperparameter Tuning

- ▶ **Parameters:** learned from data during `fit()`, e.g. regression coefficients, a tree's split points.
- ▶ **Hyperparameters:** set *before* training, control how learning happens, e.g. k in KNN, `max_depth` in a Decision Tree, C in an SVM.
- ▶ Picking by hand ("try $k = 3$, try $k = 5$, ...") doesn't scale past one hyperparameter.
- ▶ Fix: try many combinations, score each with cross validation, keep the best.

INTUITION

Parameters: what the model learns. Hyperparameters: the knobs you set before it starts learning, like oven temperature and bake time, chosen before the cake goes in.

Hyperparameter Tuning: Grid Search

Try every combination on a fixed grid, keep the best cross-validated score.

```
1 import pandas as pd
2 from sklearn.model_selection import GridSearchCV
3 from sklearn.neighbors import KNeighborsClassifier
4
5 url =
6     "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.
7 names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age',
8         'class']
9
10 df = pd.read_csv(url, names=names)
11 X, y = df.values[:, 0:8], df.values[:, 8]
12
13 model = KNeighborsClassifier()
14 param_grid = {'n_neighbors': [3, 5, 7, 9, 11, 15, 21]}
15
16 grid = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
17 grid.fit(X, y)
18
19 # Output: Best k 11, accuracy 0.749
```

Hyperparameter Tuning: Grid Search, Cost

- ▶ 7 candidate values of $k \times 5$ folds = 35 model fits.
- ▶ All run automatically, no manual loop.
- ▶ One hyperparameter is cheap. Cost multiplies fast with more: 3 hyperparameters, 10 values each, = 1000 fits.

Hyperparameter Tuning: Randomized Search

- ▶ Grid Search checks every combination: fine for one hyperparameter, explodes with more.
- ▶ RandomizedSearchCV: sample a fixed number of random combinations instead.
- ▶ Trades a guarantee of the single best combination for a large cut in compute.

```
from sklearn.model_selection import RandomizedSearchCV
2
param_dist = {'n_neighbors': list(range(1, 30))}
4 random_search = RandomizedSearchCV(model, param_dist, n_iter=5,
                                     cv=5, scoring='accuracy', random_state=42)
6 random_search.fit(X, y)
8 # Output: tried 5 of 29 possible values -> Best k 13, accuracy 0.755
```

Hyperparameter Tuning: Randomized Search, Result

- ▶ Just 5 candidates out of 29 possible values of k .
- ▶ Matched Grid Search's result ($k = 11$ vs. $k = 13$, 0.749 vs. 0.755) at a fraction of the cost.
- ▶ In practice: often close to Grid Search's answer, rarely worse in any way that matters.

Quick Check: Hyperparameter Tuning

THINK ABOUT IT

Why is it important to use cross-validated scores, not a single train/test split, when comparing hyperparameter values during Grid or Randomized Search?

Quick Check: Hyperparameter Tuning

ANSWER

- ▶ A single split can hand a hyperparameter value a “lucky” or “unlucky” test set by chance.
- ▶ Averaging over k folds gives a far more reliable estimate of how each candidate value actually generalizes.
- ▶ So the value selected as “best” is best on average, not best on one arbitrary split.

Hands-On: End-to-End Mini Pipeline

The Dataset

- ▶ Steel Plates Faults dataset (UCI): a real manufacturing quality-control problem.
- ▶ 1941 steel plates, 27 geometric/luminosity measurements each, one of 7 recorded defect types per plate.
- ▶ Task: detect K_Scratch, a scratch-type surface defect, vs. everything else.
- ▶ 27 features: too many to exhaustively search, a real test for ranking and wrapper methods.
- ▶ Goal: pandas to load and inspect, sklearn to preprocess/select/train/evaluate, a revision lap before the algorithm-by-algorithm sessions start.

Step 1: Load and Look

```

import pandas as pd

feature_names = [
    'X_Minimum', 'X_Maximum', 'Y_Minimum', 'Y_Maximum', 'Pixels_Areas',
    'X_Perimeter', 'Y_Perimeter', 'Sum_of_Luminosity', 'Minimum_of_Luminosity',
    'Maximum_of_Luminosity', 'Length_of_Conveyer', 'TypeOfSteel_A300',
    'TypeOfSteel_A400', 'Steel_Plate_Thickness', 'Edges_Index', 'Empty_Index',
    'Square_Index', 'Outside_X_Index', 'Edges_X_Index', 'Edges_Y_Index',
    'Outside_Global_Index', 'LogOfAreas', 'Log_X_Index', 'Log_Y_Index',
    'Orientation_Index', 'Luminosity_Index', 'SigmoidOfAreas',
]

fault_names = ['Pastry', 'Z_Scratch', 'K_Scratch', 'Stains',
               'Dirtiness', 'Bumps', 'Other_Faults']

df = pd.read_csv('Faults.NNA', sep=r'\s+', header=None,
                 names=feature_names + fault_names)
df['target'] = df['K_Scratch']
print(df.shape)
df.head(3)

# Output: (1941, 35)
#   X_Minimum X_Maximum Y_Minimum ... target
# 0         42         50   270900 ...      0
# 1         645         651  2538079 ...      0

```

Step 2: Missing Values and Class Balance

```
print(df.isna().sum().sum(), "missing values")
2 print(df['target'].value_counts())

4 # Output:
# 0 missing values
6 # target
# 0      1550    (no K_Scatch defect)
8 # 1       391    (K_Scatch defect)
```

Step 3: Train/Test Split

Stratify on the target so both splits keep the same 80/20 class balance.

```
1 from sklearn.model_selection import train_test_split
2
3 X = df[feature_names].values
4 y = df['target'].values
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.25, random_state=42, stratify=y)
8
9 # Output: X_train (1455, 27)   X_test (486, 27)
```

Step 4: Scale the Features

Fit the scaler on training data only, the same no-leakage rule from Data Preparation.

```
1 from sklearn.preprocessing import StandardScaler
3 scaler = StandardScaler().fit(X_train)
  X_train_s = scaler.transform(X_train)
5 X_test_s  = scaler.transform(X_test)
```

Step 5: Baseline, All 27 Features

```
1 from sklearn.linear_model import LogisticRegression
  from sklearn.metrics import accuracy_score
3
4 model_all = LogisticRegression(max_iter=5000)
5 model_all.fit(X_train_s, y_train)
  pred_all = model_all.predict(X_test_s)
7 print("Accuracy:", accuracy_score(y_test, pred_all))
9 # Output: Accuracy: 0.977
```

Step 6: Rank Features (Filter)

Same SelectKBest approach as before, now on 27 candidates instead of 8.

```
1 from sklearn.feature_selection import SelectKBest, f_classif
3 skb = SelectKBest(score_func=f_classif, k=10).fit(X_train_s, y_train)
   top10 = [feature_names[i] for i in skb.get_support(indices=True)]
5 print(top10)

7 # Output: ['Pixels_Areas', 'X_Perimeter', 'Sum_of_Luminosity',
8 #          'Minimum_of_Luminosity', 'Outside_X_Index', 'Edges_Y_Index',
9 #          'LogOfAreas', 'Log_X_Index', 'Log_Y_Index', 'SigmoidOfAreas']
```

Step 7: Retrain on the Top 10

```
1 idx = skb.get_support(indices=True)
  model_10 = LogisticRegression(max_iter=5000)
3 model_10.fit(X_train_s[:, idx], y_train)
  pred_10 = model_10.predict(X_test_s[:, idx])
5 print("Accuracy:", accuracy_score(y_test, pred_10))

7 # Output: Accuracy: 0.951
```


Step 8: Filter Result: Not a Free Lunch

- ▶ 27 features: 0.977 test accuracy.
- ▶ Filter top 10: 0.951 test accuracy.
- ▶ 17 dropped features weren't dead weight: accuracy fell by ~ 2.6 points.
- ▶ Real trade-off: fewer features for a real accuracy cost. Worth it or not depends on the deployment.

Step 9: Try a Wrapper Instead

Greedy Forward Selection, cross-validated, on all 27 features:

```
1 from sklearn.model_selection import cross_val_score, KFold
3 Xs = StandardScaler().fit_transform(X)
  kfold = KFold(n_splits=5, shuffle=True, random_state=7)
5 selected, remaining = [], list(range(27))
7 for step in range(6):
    scores = [(f, cross_val_score(LogisticRegression(max_iter=5000),
    Xs[:, selected + [f]], y, cv=kfold).mean())) for f in remaining]
9     best_f, best_s = max(scores, key=lambda t: t[1])
11    selected.append(best_f); remaining.remove(best_f)
13 # Output: 6 rounds -> Log_X_Index, Steel_Plate_Thickness, Pixels_Areas,
  #         Square_Index, X_Minimum, Orientation_Index
```

Step 10: Filter vs. Wrapper vs. Everything

Feature set	# Features	5-fold CV Accuracy
All features	27	0.975
Filter (top by F-score)	10	0.949
Wrapper (greedy forward)	6	0.963

INTUITION

6 wrapper-chosen features close most of the gap to all 27 (0.963 vs. 0.975) while clearly beating the 10-feature filter (0.963 vs. 0.949): fewer, better-combined features can rival throwing everything at the model. The filter's top 10, chosen one at a time, missed a combination the wrapper found.

Step 11: Evaluate the Final Model

```
1 from sklearn.metrics import confusion_matrix, classification_report
2
3 wrapper_cols = ['Log_X_Index', 'Steel_Plate_Thickness', 'Pixels_Areas',
4                 'Square_Index', 'X_Minimum', 'Orientation_Index']
5 idx6 = [feature_names.index(f) for f in wrapper_cols]
6
7 model_6 = LogisticRegression(max_iter=5000)
8 model_6.fit(X_train_s[:, idx6], y_train)
9 pred_6 = model_6.predict(X_test_s[:, idx6])
10 print(confusion_matrix(y_test, pred_6))
11
12 # Output: [[384  4]
13 #          [ 16 82]]
```

Step 11: Classification Report

6 features, 96% test accuracy, beating both the 10-feature filter model and coming close to the full 27-feature model, on cross validation too, not just this one split.

```
1 print(classification_report(y_test, pred_6,  
                             target_names=['no_k_scatch', 'k_scatch']))  
3  
4  
5 # Output:  
6 #           precision    recall  f1-score   support  
7 #  no_k_scatch         0.96      0.99      0.97         388  
8 #    k_scatch          0.95      0.84      0.89          98
```

Quick Check: The Pipeline

THINK ABOUT IT

Step 6 fits `SelectKBest` on `X_train_s` (the scaled training set), not the full scaled dataset. Step 9's greedy forward selection, however, scores each candidate using cross validation on *all* of `X` (via `Xs`, not `X_train_s`). Is Step 9 leaking test information in a way Step 6 avoids?

Quick Check: The Pipeline

ANSWER

- ▶ No, both are safe, but for different reasons.
- ▶ Step 6 never touches `X_test_s` at all.
- ▶ Step 9 uses every row of `X`, including future test rows, but cross validation holds out a fold at a time: no fold's held-out data influences its own score.
- ▶ Step 9's real safety net: `X_test_s` from Step 3 was never part of its cross validation either, so Step 11's evaluation is still a fair, unseen check on the wrapper-selected features.

Try It Yourself

- ▶ Change $k=10$ to $k=5$ in Step 6: does accuracy hold up with even fewer features?
- ▶ Run Greedy Backward Elimination instead of Forward on all 27 features: same 6 features at the end?
- ▶ Swap `LogisticRegression` for a different classifier: does the winning feature set change?
- ▶ Try a different target fault type (e.g. Bumps): does the same feature set still win? (Spoiler: no, worth discussing why.)

Recap: The Pipeline Pattern

- ▶ Load → Inspect → Split → Scale → Select → Train → Evaluate → Compare.
- ▶ Every step reused today: pandas for loading/inspecting, sklearn for the rest.
- ▶ This shape doesn't change once you swap in a real ML algorithm: only Step 5/7's model does.
- ▶ Next: the algorithms themselves, one at a time, starting with Linear Regression.

Thanks ...

- ▶ Office Hours: Saturdays, 3 to 5 pm (IST); Free-Open to all; email for appointment to yogeshkulkarni at yahoo dot com
- ▶ Call + 9 1 9 8 9 0 2 5 1 4 0 6



(<https://www.linkedin.com/in/yogeshkulkarni/>)



(<https://medium.com/@yogeshharibhaukulkarni>)



(<https://www.github.com/yogeshhk/>)